

Лекция 5: Одномерные и двумерные массивы

Введение

До сих пор мы работали с отдельными переменными: одно число, одна строка, одно логическое значение. Но в реальных программах часто приходится работать с наборами данных: список студентов, таблица результатов, коллекция товаров в корзине. Для хранения таких наборов однотипных данных в Java используются **массивы**.

Массив — это контейнер фиксированного размера, который хранит элементы одного типа. Каждый элемент имеет свой номер — **индекс**. Индексация в Java начинается с **0**.

Сегодня мы изучим:

- Одномерные массивы: создание, заполнение, перебор, типовые алгоритмы
- Двумерные массивы: таблицы, матрицы, работа с рядами и столбцами
- Особенности массивов в Java (фиксированный размер, связь с памятью)

1) Одномерные массивы

Объявление и создание массива

Существует несколько способов объявить и создать массив:

```
public class ArrayDeclaration {
    public static void main(String[] args) {
        // Способ 1: объявление + выделение памяти (без заполнения)
        int[] numbers1 = new int[5];    // массив из 5 целых чисел (все элементы = 0)

        // Способ 2: объявление и заполнение
        int[] numbers2 = {10, 20, 30, 40, 50}; // массив из 5 элементов с указанными значениями

        // Способ 3: через new с заполнением
        int[] numbers3 = new int[]{1, 2, 3, 4, 5};

        // Альтернативный синтаксис (не рекомендуется, но встречается)
        int numbers4[] = new int[3]; // C-стиль, лучше не использовать
    }
}
```

```
}
```

Важные правила:

- Размер массива указывается при создании и **не может быть изменён** позже
- Если массив создан через `new int[5]`, то элементы получают значения по умолчанию:
 - Для числовых типов: 0
 - Для `boolean`: `false`
 - Для ссылочных типов (включая `String`): `null`

Доступ к элементам массива

Доступ к элементу осуществляется по индексу в квадратных скобках. Индексы от 0 до `длина_массива - 1`.

```
public class ArrayAccess {
    public static void main(String[] args) {
        int[] numbers = {10, 20, 30, 40, 50};

        // Получение элемента по индексу
        System.out.println("Первый элемент: " + numbers[0]); // 10
        System.out.println("Третий элемент: " + numbers[2]); // 30
        System.out.println("Последний элемент: " + numbers[4]); // 50

        // Изменение элемента
        numbers[1] = 25;
        System.out.println("Изменённый второй элемент: " + numbers[1]); // 25

        // ОШИБКА: выход за границы массива (индекс 5 не существует)
        // System.out.println(numbers[5]); // ArrayIndexOutOfBoundsException
    }
}
```

Длина массива (свойство `length`)

Каждый массив имеет свойство `length`, которое возвращает количество элементов:

```
public class ArrayLength {
    public static void main(String[] args) {
        int[] numbers = {10, 20, 30, 40, 50};
        System.out.println("Длина массива: " + numbers.length); // 5
    }
}
```

```

// Обычно length используется в циклах для перебора
for (int i = 0; i < numbers.length; i++) {
    System.out.println("numbers[" + i + "] = " + numbers[i]);
}
}
}

```

Перебор массива: цикл **for-each**

Начиная с Java 5, существует улучшенный цикл **for-each** (также называется "for в стиле for-each"), который упрощает перебор всех элементов массива, когда индекс не нужен.

```

public class ForEachExample {
    public static void main(String[] args) {
        int[] numbers = {10, 20, 30, 40, 50};

        // Классический for
        System.out.println("Классический for:");
        for (int i = 0; i < numbers.length; i++) {
            System.out.print(numbers[i] + " ");
        }

        // For-each (только для чтения)
        System.out.println("\nFor-each:");
        for (int value : numbers) {
            System.out.print(value + " ");
        }

        // For-each нельзя использовать для изменения элементов
        for (int value : numbers) {
            value = value * 2; // это не изменит массив!
        }
        System.out.println("\nПосле попытки изменения через for-each:");
        for (int value : numbers) {
            System.out.print(value + " "); // значения не изменились
        }
    }
}

```

Когда использовать **for-each**:

- Когда нужно просто прочитать все элементы массива

- Когда не требуется индекс элемента

Когда использовать классический for:

- Когда нужно изменить элементы массива
- Когда нужен доступ к индексу
- Когда нужно пройти только часть массива

Типовые алгоритмы с массивами

Сумма всех элементов

```
public class ArraySum {
    public static void main(String[] args) {
        int[] numbers = {5, 10, 15, 20, 25};
        int sum = 0;

        for (int i = 0; i < numbers.length; i++) {
            sum += numbers[i];
        }

        System.out.println("Сумма элементов: " + sum); // 75
        System.out.println("Среднее арифметическое: " + (double)sum / numbers.length); // 15.0
    }
}
```

Поиск максимального и минимального элемента

```
public class ArrayMinMax {
    public static void main(String[] args) {
        int[] numbers = {12, 45, 7, 89, 23, 56, 3, 78};

        // Предполагаем, что первый элемент — искомый
        int max = numbers[0];
        int min = numbers[0];

        for (int i = 1; i < numbers.length; i++) {
            if (numbers[i] > max) {
                max = numbers[i];
            }
            if (numbers[i] < min) {
                min = numbers[i];
            }
        }
    }
}
```

```

    }

    System.out.println("Максимальный элемент: " + max); // 89
    System.out.println("Минимальный элемент: " + min); // 3
}
}

```

Поиск элемента по значению

```

public class ArraySearch {
    public static void main(String[] args) {
        int[] numbers = {12, 45, 7, 89, 23, 56, 3, 78};
        int target = 23;
        int index = -1; // -1 означает "не найден"

        for (int i = 0; i < numbers.length; i++) {
            if (numbers[i] == target) {
                index = i;
                break; // нашли — выходим
            }
        }

        if (index != -1) {
            System.out.println("Элемент " + target + " найден на позиции " + index);
        } else {
            System.out.println("Элемент " + target + " не найден");
        }
    }
}

```

Реверс массива (перестановка элементов в обратном порядке)

```

public class ArrayReverse {
    public static void main(String[] args) {
        int[] numbers = {1, 2, 3, 4, 5, 6};

        System.out.println("Исходный массив:");
        for (int num : numbers) {
            System.out.print(num + " ");
        }

        // Реверс: меняем местами первый с последним, второй с предпоследним
        // и т.д.
        for (int i = 0; i < numbers.length / 2; i++) {

```

```

    int temp = numbers[i];
    numbers[i] = numbers[numbers.length - 1 - i];
    numbers[numbers.length - 1 - i] = temp;
}

System.out.println("\nМассив после реверса:");
for (int num : numbers) {
    System.out.print(num + " ");
}
}
}

```

Копирование массива

```

public class ArrayCopy {
    public static void main(String[] args) {
        int[] source = {1, 2, 3, 4, 5};

        // Способ 1: ручное копирование через цикл
        int[] copy1 = new int[source.length];
        for (int i = 0; i < source.length; i++) {
            copy1[i] = source[i];
        }

        // Способ 2: метод System.arraycopy()
        int[] copy2 = new int[source.length];
        System.arraycopy(source, 0, copy2, 0, source.length);

        // Способ 3: метод Arrays.copyOf() (нужен import java.util.Arrays)
        int[] copy3 = java.util.Arrays.copyOf(source, source.length);

        // ВАЖНО: простое присваивание НЕ создаёт копию!
        int[] notCopy = source; // не копия, а ссылка на тот же массив
        notCopy[0] = 100;
        System.out.println("source[0] после изменения через ссылку: " + source[0])
; // 100!
    }
}

```

2) Двумерные массивы

Двумерный массив в Java — это массив массивов. Его можно представить как таблицу с **рядами** (первое измерение) и **столбцами** (второе измерение).

Объявление и создание двумерного массива

```
public class TwoDimDeclaration {
    public static void main(String[] args) {
        // Способ 1: матрица 3×4 (3 строки, 4 столбца), все элементы = 0
        int[][] matrix1 = new int[3][4];

        // Способ 2: заполнение при объявлении
        int[][] matrix2 = {
            {1, 2, 3, 4},
            {5, 6, 7, 8},
            {9, 10, 11, 12}
        };

        // Способ 3: через new с указанием значений
        int[][] matrix3 = new int[][]{
            {1, 2, 3},
            {4, 5, 6}
        };

        // "Рваный" массив: строки могут иметь разную длину
        int[][] jagged = new int[3][];
        jagged[0] = new int[]{1, 2};
        jagged[1] = new int[]{3, 4, 5, 6};
        jagged[2] = new int[]{7, 8, 9};
    }
}
```

Доступ к элементам и перебор

```
public class TwoDimAccess {
    public static void main(String[] args) {
        int[][] matrix = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };

        // Доступ к элементу: matrix[строка][столбец]
        System.out.println("Элемент во 2-й строке, 3-м столбце: " + matrix[1][2]); /

        // Изменение элемента
    }
}
```

```
matrix[0][0] = 100;
```

```
// Перебор с помощью вложенных циклов
```

```
System.out.println("Перебор через for:");
```

```
for (int i = 0; i < matrix.length; i++) { // по строкам
    for (int j = 0; j < matrix[i].length; j++) { // по столбцам
        System.out.print(matrix[i][j] + " ");
    }
    System.out.println(); // переход на новую строку
}
```

```
// Перебор через for-each
```

```
System.out.println("Перебор через for-each:");
```

```
for (int[] row : matrix) { // для каждой строки
    for (int value : row) { // для каждого элемента в строке
        System.out.print(value + " ");
    }
    System.out.println();
}
}
```

Типовые алгоритмы с двумерными массивами

Сумма всех элементов

```
public class TwoDimSum {
    public static void main(String[] args) {
        int[][] matrix = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };

        int totalSum = 0;
        for (int i = 0; i < matrix.length; i++) {
            for (int j = 0; j < matrix[i].length; j++) {
                totalSum += matrix[i][j];
            }
        }
        System.out.println("Сумма всех элементов: " + totalSum); // 45
    }
}
```


Сумма по каждой строке

```
public class RowSum {
    public static void main(String[] args) {
        int[][] matrix = {
            {1, 2, 3},
            {4, 5, 6},
            {7, 8, 9}
        };

        for (int i = 0; i < matrix.length; i++) {
            int rowSum = 0;
            for (int j = 0; j < matrix[i].length; j++) {
                rowSum += matrix[i][j];
            }
            System.out.println("Сумма строки " + i + ": " + rowSum);
        }
    }
}
```

Транспонирование матрицы (строки и столбцы меняются местами)

```
public class TransposeMatrix {
    public static void main(String[] args) {
        int[][] matrix = {
            {1, 2, 3},
            {4, 5, 6}
        };
        // Исходная: 2 строки, 3 столбца
        // Транспонированная: 3 строки, 2 столбца

        int rows = matrix.length;
        int cols = matrix[0].length;
        int[][] transposed = new int[cols][rows];

        for (int i = 0; i < rows; i++) {
            for (int j = 0; j < cols; j++) {
                transposed[j][i] = matrix[i][j];
            }
        }

        System.out.println("Транспонированная матрица:");
        for (int i = 0; i < transposed.length; i++) {
            for (int j = 0; j < transposed[i].length; j++) {
```

```

        System.out.print(transposed[i][j] + " ");
    }
    System.out.println();
}
}
}

```

Поиск максимального элемента и его позиции

```

public class TwoDimMax {
    public static void main(String[] args) {
        int[][] matrix = {
            {12, 45, 7},
            {89, 23, 56},
            {3, 78, 34}
        };

        int max = matrix[0][0];
        int maxRow = 0, maxCol = 0;

        for (int i = 0; i < matrix.length; i++) {
            for (int j = 0; j < matrix[i].length; j++) {
                if (matrix[i][j] > max) {
                    max = matrix[i][j];
                    maxRow = i;
                    maxCol = j;
                }
            }
        }

        System.out.println("Максимальный элемент: " + max);
        System.out.println("Находится в строке " + maxRow + ", столбце " + maxCol);
    }
}

```

3) Особенности массивов в Java

Массивы — это объекты

В Java массив является объектом. Он имеет свои поля (например, `length`) и методы (например, `clone()`). Массивы хранятся в куче (heap).

```

public class ArrayIsObject {
    public static void main(String[] args) {
        int[] arr = {1, 2, 3};
        System.out.println("Тип массива: " + arr.getClass().getName()); // [I
        System.out.println("toString(): " + arr.toString()); // не читаемый вывод

        // Для читаемого вывода используем Arrays.toString()
        System.out.println("Arrays.toString(): " + java.util.Arrays.toString(arr));
    }
}

```

Передача массива в метод

Массив передаётся по ссылке — изменения внутри метода влияют на исходный массив.

```

public class ArrayMethod {
    public static void main(String[] args) {
        int[] numbers = {1, 2, 3, 4, 5};
        multiplyByTwo(numbers);

        for (int num : numbers) {
            System.out.print(num + " "); // 2 4 6 8 10
        }
    }

    public static void multiplyByTwo(int[] arr) {
        for (int i = 0; i < arr.length; i++) {
            arr[i] *= 2;
        }
    }
}

```

Сортировка массива (встроенный метод)

```

import java.util.Arrays;

public class ArraySort {
    public static void main(String[] args) {
        int[] numbers = {42, 15, 7, 89, 23, 5};

        System.out.println("До сортировки: " + Arrays.toString(numbers));
        Arrays.sort(numbers);
    }
}

```

```
System.out.println("После сортировки: " + Arrays.toString(numbers));

// Для строк сортировка лексикографическая
String[] fruits = {"banana", "apple", "cherry", "date"};
Arrays.sort(fruits);
System.out.println("Строки после сортировки: " + Arrays.toString(fruits));
}
```

Итоги пятой лекции

Сегодня мы изучили:

Одномерные массивы:

- Объявление: `int[] arr = new int[5];` или `int[] arr = {1,2,3};`
- Доступ по индексу: `arr[index]` (индексы от 0 до `length-1`)
- Свойство `length` — длина массива
- Перебор: классический `for` и `for-each`
- Типовые алгоритмы: сумма, поиск `min/max`, реверс, копирование

Двумерные массивы:

- Объявление: `int[][] matrix = new int[3][4];`
- Представление как "массив массивов" (могут быть "рваные" массивы)
- Доступ: `matrix[row][col]`
- Вложенные циклы для перебора
- Типовые алгоритмы: сумма по строкам, транспонирование, поиск максимума

Ключевые выводы:

1. Размер массива фиксирован при создании и не может измениться
2. Индексация начинается с 0, последний индекс = `length - 1`
3. Выход за границы вызывает `ArrayIndexOutOfBoundsException`
4. Массивы являются объектами и передаются по ссылке
5. Для удобной работы с массивами используйте `java.util.Arrays`